Calidad del Software Actividad: Caso Estudio Integrador.

Desarrollo de Calculadora usando metodología RUP.

Disciplina: 1. Requisitos.

Estudiante: Diego Andrés Díaz.

Cc 80032909

Docente: Ing Leonel Andrés Lópe Arias.

Institución: Politécnico Internacional.

Fecha: Noviembre 2026.

1. Requisitos Artefactos de la disciplina de Requisitos.

Proyecto Calculadora MVC con RUP (prototipo del Sprint 1)

Empresa Edmsoft Fase / iteración Transición / T1

Versión y fecha 1.0 – 1 de noviembre de 2026

Rol Desarrollador Fullstack Integrante Diego Andrés Díaz.

2. Introducción

Para el caso estudio integrador, Edmsoft entrega en el primer sprint un prototipo de calculadora con interfaz gráfica, construido con la metodología RUP y la arquitectura MVC (modelo Vista Controlador).

Este documento reúne los artefactos de la disciplina de Requisitos, el prototipo cubre la suma y la multiplicación, la resta y la división quedan registradas como características candidatas para una iteración posterior (Como aporte adicional al trabajo y mostrando un factor de diferenciación a lo solicitado, es decir aportando al ejercicio).

Trabajo a realizar (Tabla 1 del caso)	Artefacto resultante	Sección
Enumerar los requisitos candidatos	Lista de características	2
Comprender el contexto del sistema	Modelo del dominio	3
Capturar los requisitos funcionales	Modelo de casos de uso	4 y 5
Capturar los requisitos no funcionales	Requisitos adicionales	6

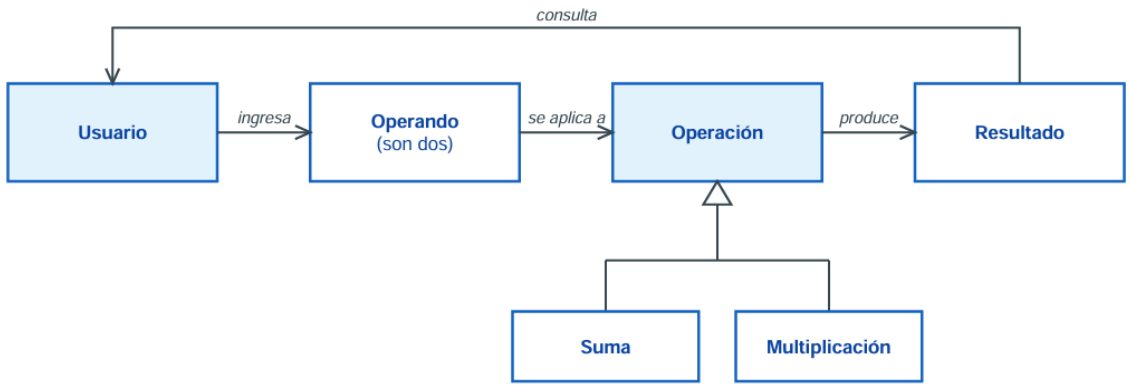
3. Lista de características.

Las características son las funciones que el cliente espera del sistema, ordenadas por prioridad.

ID	Característica	Prioridad	Estado en la transición
CAR-01	Ingresar dos datos numéricos en un formulario	Alta	Implementada
CAR-02	Sumar los dos datos con el botón «sumar»	Alta	Implementada
CAR-03	Multiplicar los dos datos con el botón «multiplicar»	Alta	Implementada
CAR-04	Mostrar el resultado en otra sección del mismo formulario	Alta	Implementada
CAR-05	Validar entradas no numéricas y resultados fuera de rango	Media	Implementada
CAR-06	Estructurar el sistema con el patrón MVC (vista, controlador, modelo)	Alta	Implementada
CAR-07	Restar dos datos	Baja	Candidata (iteración posterior)
CAR-08	Dividir dos datos controlando la división por cero	Baja	Candidata (iteración posterior)

4. Modelo de dominio.

El modelo del dominio describe los conceptos del problema, sin hablar todavía de clases de software. Un Usuario ingresa dos Operandos; una Operación (suma o multiplicación) se aplica a ellos y produce un Resultado que el usuario consulta.



Modelo de dominio, la resta y la división se agregarán como aporte adicional al caso estudio integrador.

Concepto	Descripción
Usuario	Persona que usa el formulario para obtener el resultado de una operación.
Operando	Cada uno de los dos valores numéricos (enteros, decimales o negativos) que se ingresan.
Operación	Regla matemática aplicada a los operandos. Especializaciones actuales: Suma y Multiplicación.
Resultado	Valor numérico obtenido, o mensaje que informa que los datos no son válidos.

5. Modelo de casos de uso.

Hay un solo actor, el Usuario. Los casos de uso «Sumar» y «Multiplicar» incluyen la validación de los datos.

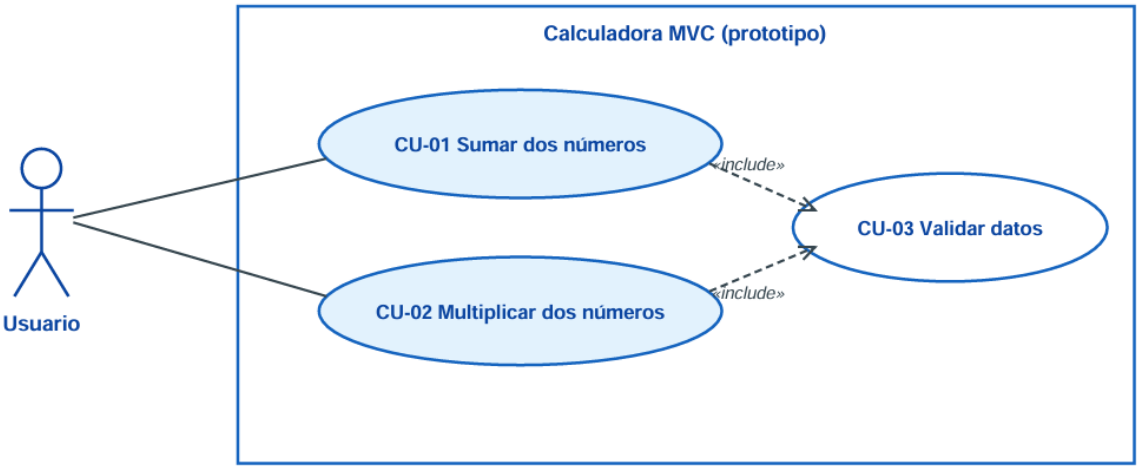


Figura 2. Diagrama de casos de uso del prototipo.

Especificación de los casos de uso.

Campo	CU-01 Sumar dos números
Actor	Usuario
Precondición	La aplicación está abierta y el formulario visible.
Flujo principal	1. El usuario escribe el dato 1. 2. El usuario escribe el dato 2. 3. El usuario pulsa «sumar». 4. El sistema ejecuta CU-03 (validar datos). 5. El sistema calcula dato 1 + dato 2. 6. El sistema muestra el resultado en el campo «resultado».
Flujo alternativo	4a. Algún dato no es un número válido, o el cálculo sale del rango: el sistema muestra un mensaje en «resultado» y el usuario puede corregir los datos y repetir desde el paso 3.
Postcondición	Se visualiza el resultado correcto o un mensaje de error. La ventana sigue abierta.
Requisitos	RF-01, RF-02, RF-04, RF-05

Campo	CU-02 Multiplicar dos números
Actor	Usuario
Precondición	La aplicación está abierta y el formulario visible.
Flujo principal	1. El usuario escribe el dato 1 y el dato 2. 2. El usuario pulsa «multiplicar». 3. El sistema ejecuta CU-03 (validar datos). 4. El sistema calcula dato 1 x dato 2. 5. El sistema muestra el resultado en el campo «resultado».
Flujo alternativo	3a. Datos no válidos o resultado fuera de rango: se muestra un mensaje y el usuario corrige.
Postcondición	Se visualiza el resultado correcto o un mensaje de error.
Requisitos	RF-01, RF-03, RF-04, RF-05

Campo	CU-03 Validar datos (incluido en CU-01 y CU-02)
Descripción	Comprueba que ambos datos se puedan convertir a número y que el resultado sea un valor finito.
Mensajes	«Ingrese números válidos» (texto, campo vacío) y «Valores fuera de rango» (inf, nan o desbordamiento).

6. Requisitos funcionales.

ID	Requisito	Criterio de aceptación	Caso de uso
RF-01	El formulario permite ingresar dos datos.	Existen dos campos rotulados «dato 1» y «dato 2» que aceptan enteros, decimales y negativos.	CU-01, CU-02
RF-02	El sistema suma los datos al pulsar «sumar».	5 y 3 muestran 8. -2 y 0.5 muestran -1.5.	CU-01
RF-03	El sistema multiplica los datos al pulsar «multiplicar».	1000 y 34 muestran 34000.	CU-02
RF-04	El resultado se muestra en otra sección del mismo formulario.	El resultado aparece en el campo «resultado», que es de solo lectura.	CU-01, CU-02
RF-05	El sistema informa las entradas inválidas sin cerrarse.	Con «hola» o un campo vacío muestra «Ingrese números válidos». Con «inf», «nan» o un desbordamiento muestra «Valores fuera de rango».	CU-03

7. Requisitos no funcionales.

ID	Categoría	Requisito	Forma de verificación
RNF-01	Usabilidad	Un solo formulario con etiquetas claras y paleta azul (#1565c0) y blanco (#ffffff).	Inspección visual
RNF-02	Rendimiento	La operación es local y el resultado aparece de inmediato.	Medición del tiempo de ejecución
RNF-03	Mantenibilidad	La lógica de negocio no depende de la interfaz (MVC).	Pruebas del modelo sin interfaz
RNF-04	Portabilidad	Python 3 con Tkinter, sin librerías externas.	Ejecución en el entorno documentado
RNF-05	Simplicidad	Código corto y legible, con sintaxis simple.	Revisión de código

8. Trazabilidad.

La trazabilidad relaciona cada requisito con el caso de uso que lo realiza, la clase que lo implementa y las pruebas que lo verifican (los casos CP se describen en el documento de Pruebas).

Requisito	Caso de uso	Clase responsable	Casos de prueba
RF-01	CU-01, CU-02	Vista	CP-13, CP-14
RF-02	CU-01	Calculadora (modelo), Controlador	CP-01 a CP-04, CP-13
RF-03	CU-02	Calculadora (modelo), Controlador	CP-05 a CP-07, CP-14
RF-04	CU-01, CU-02	Vista	CP-13, CP-14, CP-15
RF-05	CU-03	Controlador	CP-08 a CP-12, CP-15

Los requisitos RF-01 a RF-05 y RNF-01 a RNF-05 están implementados y verificados (ver documento de Pruebas).

No hay cambios de alcance pendientes. Las características CAR-07 (restar) y CAR-08 (dividir) quedan como backlog.

Durante la transición se endureció RF-05: se añadió el mensaje «Valores fuera de rango» tras el defecto DEF-01.